Image Segmentation Using Max-Flow and Min-Cut

Nikhil and Shashank

Flow Networks

- A flow network is a directed graph with:
  - capacities on edges
  - a source (s)
  - a sink (t)
- Represents movement of some quantity.
- Capacity rule:

$$0 \leq f(u, v) \leq c(u, v)$$

Flow and Max-Flow

- **Flow:** amount sent through edges.
- Must satisfy:
  - capacity constraint
  - flow conservation:

$$\sum_{\text{incoming}} f = \sum_{\text{outgoing}} f$$

- **Max-flow:** maximum possible flow from s to t.

Cut and Min-Cut

- A **cut** divides nodes into:

$$(S, T), \quad s \in S, \ t \in T$$

- Cut capacity:

$$C(S, T) = \sum_{u \in S, v \in T} c(u, v)$$

- **Min-cut:** cut with smallest capacity.

Max-Flow Min-Cut Theorem

- Maximum flow value equals minimum cut capacity.
- Any flow $\leq$ capacity of any cut.
- When no augmenting path exists, min-cut is found.

Ford–Fulkerson Algorithm

- Uses augmenting paths in residual graph.
- Key terms:
  - residual capacity
  - augmenting path
  - bottleneck
- Repeats until no s–t path remains.
- **Time Complexity:**

$$O(E \times F)$$

- Not guaranteed to terminate for irrational capacities.

Ford–Fulkerson Pseudocode

```
FordFulkerson(G, s, t):
    for every edge (u, v):
        flow(u, v) = 0

    while there exists augmenting path P:
        bottleneck = min residual capacity on P
        for each edge (u, v) in P:
            if forward edge: increase flow
            else backward edge: decrease flow
        update residual graph

    return max flow
```

Edmonds–Karp Improvement

- Uses BFS to find shortest augmenting path.
- Guaranteed polynomial runtime.
- **Time Complexity:**

$$O(VE^2)$$

- Faster and more stable for image segmentation.

Image Segmentation Basics

- Divide image into meaningful regions.
- Applications:
  - medical imaging
  - object extraction
  - autonomous driving
- Approaches:
  - thresholding
  - clustering
  - deep learning
  - graph-based segmentation

Graph-Cut Segmentation Steps

- Convert pixels into graph nodes.
- Add source (FG) and sink (BG).
- Add:
  - N-links (pixel–pixel)
  - T-links (pixel–terminal)
- Run max-flow / min-cut.

Assigning Capacities

N-links:

$$c(p,q) = \lambda \exp\left(-\frac{(I_p - I_q)^2}{2\sigma^2}\right)$$

T-links:

$$c(p \to s) = -\log P(FG|I_p) \quad c(p \to t) = -\log P(BG|I_p)$$

Using Min-Cut

- After max-flow:
  - source side $\Rightarrow$ foreground
  - sink side $\Rightarrow$ background
- Produces globally optimal boundary.

Advantages and Disadvantages

Advantages:

- Globally optimal segmentation.
- Robust to noise.
- Works without training data.
- Supports user constraints.

Disadvantages:

- Slow for large images.
- Sensitive to parameter tuning.
- Primarily binary segmentation.
- Memory intensive for big graphs.

Possible Improvements

- Use Edmonds–Karp or Boykov–Kolmogorov (faster).
- Switch to 8-connected neighbors.
- Better T-links using probability models (GMM).
- Allow user-marked foreground/background seeds.
- Precompute exponential lookup table for speed.
- Avoid recursive DFS to prevent stack overflow.

Thank You!